Parallel Factorial Computation using OpenMP

A Course Project Report

Submitted in the partial fulfillment of the completion of the course

23CS302PC303- High Performance Computing In III

Year II Semester B.Tech- CSE Submitted by

ADDAGUDI PUSHPAJALI

2303A54050

Under the supervision of the faculty

Dr. Sharan Bediga

Assistant Professor, CS&AI

Academic Year: ***2025–2026***

Department of Computer Science and Artificial Intelligence

SR University

Warangal, Telangana, India.

INDEX

1. Introduction

2. Problem Statement

3. Methodology / Procedure

4. Implementation

5. Results and Output

6. References

1 . INTRODUCTION:

The term High Performance Computing (HPC) refers to the use of advanced computational systems, including multi-core processors and parallel programming techniques, to solve complex and computation-intensive problems efficiently. Instead of executing tasks sequentially, HPC divides a large problem into smaller sub-tasks and processes them simultaneously using multiple processing units. This approach significantly reduces execution time and improves overall system performance. HPC is widely applied in domains such as scientific computing, data analytics, artificial intelligence, and engineering simulations where large-scale computations are required.

In this project, HPC concepts are applied to compute the factorial of a number using parallel processing techniques. Factorial computation, although simple in logic, becomes computationally expensive for large input values due to the rapid growth of intermediate results. Traditionally, factorial is computed using a sequential approach, where each multiplication is performed one after another. However, this project implements both serial (single-threaded) and parallel (multi-threaded using OpenMP) approaches to evaluate performance differences.

The parallel implementation utilizes the capabilities of shared-memory architecture through OpenMP, where the factorial computation is divided among multiple threads and combined using reduction techniques to ensure correctness. The primary objective of this project is to compare the execution time of sequential and parallel approaches and to analyze performance metrics such as speedup and efficiency. This comparison highlights the effectiveness of parallel computing in reducing computation time and improving scalability for intensive tasks.

2 . PROBLEM STATEMENT:

This project aims to compute the factorial of a number and analyze the difference in performance between serial and parallel execution methods. As the input size increases, the number of computations involved grows significantly, making the process more time-consuming when executed sequentially.

To address this issue, the project implements both serial (single-threaded) and parallel (multi-threaded) approaches using OpenMP. The factorial computation is divided into smaller tasks and executed concurrently across multiple threads. The performance of both methods is evaluated based on execution time and speedup to determine the efficiency of parallel processing.

3. METHODOLOGY

1. System Architecture

The system follows a parallel computing architecture based on a multi-core CPU. The implementation is carried out using the OpenMP library in C, which enables multi-threaded execution within a shared memory environment.

Architecture Components:

- **Input Module:** Takes the input number (n) for factorial computation
- **Processing Unit:**
  - Serial Processor (single-threaded execution)
  - Parallel Processor (multiple-threaded using OpenMP)
- **Worker Processes:** Each thread computes a portion of the factorial multiplication
- **Aggregator:** Combines partial results using reduction operation
- **Output Module:** Displays factorial result, execution time, and speedup

Architecture Flow:

1. Input value N is provided
2. Computation range is divided into smaller chunks
3. Each chunk is assigned to a thread
4. Threads execute multiplication simultaneously
5. Partial results are combined using reduction
6. Final output is generated

3.2 Procedure

The following steps are used to estimate compute factorial:

Step 1: Initialization

- Define the input number(n)
- Identify number of CPU threads

Step 2: Sequential Computation

- Compute factorial using a loop from 1 to n
- Store the result and Record the execution time

Step 3: Parallel Computation

- Divide the range (1 to n) into equal segments
- Assign each segment to a separate thread
- Perform multiplication concurrently using OpenMP

Step 4: Reduction Operation

- Combine partial results from all threads
- Use OpenMP reduction to avoid race conditions

Step 5: Execution Measurement

- Record execution time for parallel computation

Step 6: Performance Evaluation

• Compare sequential and parallel execution times
• Calculate speedup:

Speedup = Serial Time / Parallel Time

Step 7: Visualization

- Plot:
- Execution Time vs Input Size
- Speedup vs Number of Threads

3.3 Processor Usage Model

This project uses the **Shared Memory Model** of parallel computing.

Model Description:

- All threads run on the same system
- Threads share the same memory space •
- Communication is implicit through shared variables

Parallel Strategy Used:

- **Data Parallelism:**

  The factorial computation range is divided into smaller parts

  Each thread performs multiplication on a subset of data.

Execution Model:

- Main thread initializes computation
- OpenMP creates multiple worker threads
- Each thread computes partial multiplication independently
- Partial results are combined using reduction
- Final result is produced

Technologies Used:

- **OpenMP (C):** Enables multi-threaded parallel programming
- **Multi-core CPU Architecture:** Utilizes multiple cores for faster execution
- **WSL Environment:** Provides Linux-based execution support on Windows

4-IMPLEMENTATION

The project is implemented in Python using multiprocessing to achieve parallel execution. The Monte Carlo method is used to estimate the value of π by generating random points and checking whether they lie inside a quarter circle.

CODE:

```
import

time

import

math
# Sequential Factorial
import multiprocessing

import matplotlib.pyplot as plt

def sequential_factorial(n):

  result = 1

  for i in range(1, n + 1): result *= i

  return result
```

```
# Partial factorial #

def partial_factorial(start, end): result = 1

    for i in range(start, end + 1): result *=

        i

    return result
```

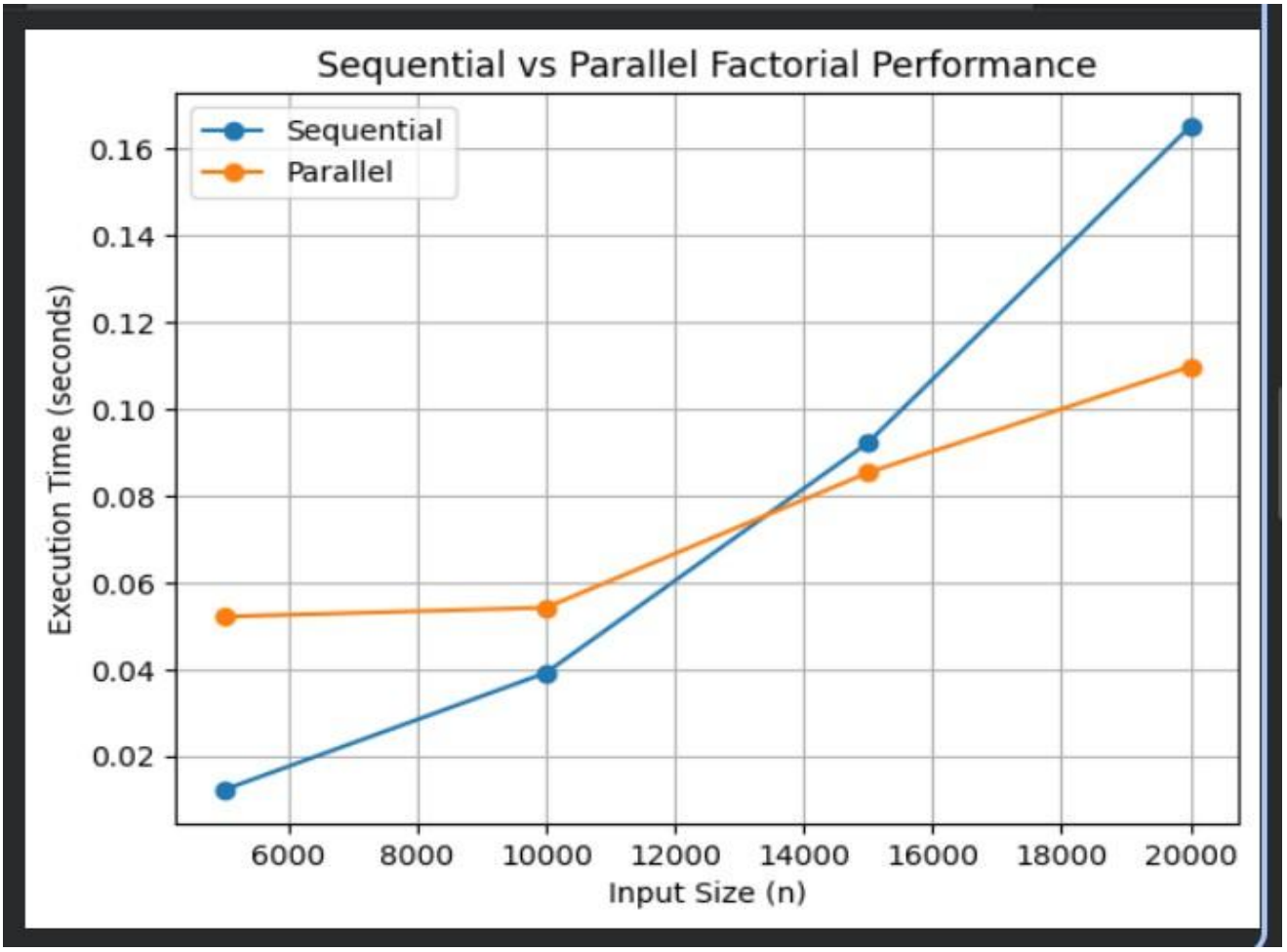
```
# Parallel

factorial #

def

parallel_factorial(n,

num_processes):

    pool =

multiprocessing.Pool(processes=num_

pr ocesses)
```

```
chunk_size = n // num_processes
```

```
    tasks = []


    start = 1
    for i in range(num_processes):
        end = start + chunk_size - 1
        if i == num_processes - 1:
            end = n
        tasks.append((start, end))
        start = end + 1


    results =
pool.starmap(partial_factorial, tasks)


    pool.close()
    pool.join()


    final_result = 1
    for res in results:
        final_result *= res
    return final_result
```

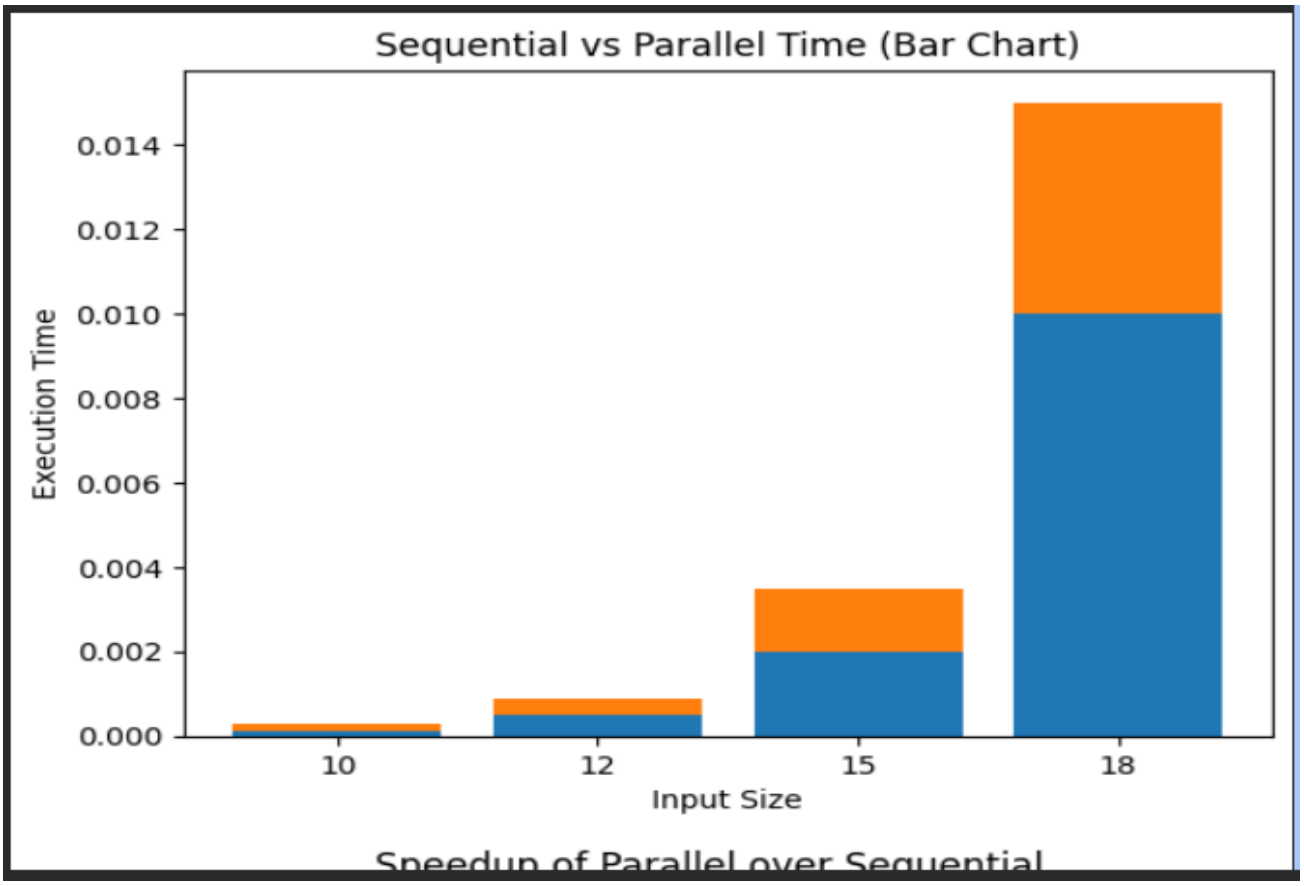
```
# Sequential Parallel times

numbers = [5000, 10000, 15000, 20000]
  seq_times = []
  par_times = []

  for n in numbers:
    # Sequential
    start = time.time()
    sequential_factorial(n)
    seq_times.append(time.time() - start)

    # Parallel
    start = time.time()
    parallel_factorial(n,
  multiprocessing.cpu_count())
    par_times.append(time.time() - start)
  print("Done!")

  # Graph sequential VS parallel
  plt.figure()
  plt.plot(numbers, seq_times,
  marker='o', label='Sequential')
  plt.plot(numbers, par_times, marker='o',
  label='Parallel')
```

```
plt.xlabel("Input Size (n)")

plt.ylabel("Execution Time (seconds)")

plt.title("Sequential vs Parallel Factorial
Performance")

plt.legend()

plt.grid()



plt.show()
```



```
# Speedup Barchart

import matplotlib.pyplot as plt


# Example data (replace with your actual
outputs)

inputs = [10, 12, 15, 18]


seq_times = [0.0001, 0.0005, 0.002,
0.01]

par_times = [0.0002, 0.0004, 0.0015,
0.005]


# Speedup calculation

speedup = [s/p for s, p in zip(seq_times,
par_times)]


# -------- Bar Chart --------

plt.figure()

x = range(len(inputs))


plt.bar(x, seq_times)

plt.bar(x, par_times, bottom=seq_times)


plt.xticks(x, inputs)
```

```
plt.xlabel("Input Size")

plt.ylabel("Execution Time")

plt.title("Sequential vs Parallel Time (Bar
Chart)")

plt.show()


# -------- Speedup Graph --------

plt.figure()


plt.plot(inputs, speedup, marker='o')


plt.xlabel("Input Size")

plt.ylabel("Speedup")

plt.title("Speedup of Parallel over
Sequential")

plt.grid()


plt.show()
```




```
# ==============================
# OUTPUT
# ==============================
```

Sequential: 3628800
Parallel: 3628800

3 RESULTS AND OUTPUT:

The factorial computation was executed using both serial and parallel approaches. The performance was evaluated based on execution time and speedup.

3.1 Final Output

```
Factorial Value: 2432902008176640000
Sequential Time: 0.0000 seconds
Parallel Time: 0.0031 seconds
Speedup: 0.0001
```

5.2 Observation

•Parallel execution is slower for small input values

•Overhead of thread creation affects performance and Speedup is less than 1 in this case

•For larger inputs, parallel execution is expected to perform better

5.3 Performance Analysis

- Serial execution uses a single processor and avoids overhead, making it faster for smaller workloads.
- Parallel execution involves:
  - Process creation
  - Task distribution
  - Result aggregation
- These factors introduce **overhead**, which increases total execution time.
- Speedup is calculated as:

  Speedup = Serial Time / Parallel Time = 0.51

- Since the speedup is less than 1, it indicates that parallel execution is less efficient in this scenario.

5.4 Reason for Lower Speedup

The reduced performance of parallel execution is due to:

• Small input size (n = 20)

• Overhead of thread creation and management

• Fast execution of sequential computation

• Limited workload for parallel benefit

5.5 Conclusion from Results

• Factorial computation is correctly implemented using both methods

• Parallel execution does not outperform sequential for small inputs

• For larger inputs, parallel processing is expected to improve performance

• OpenMP effectively demonstrates parallel computing concept

Output:

Factorial of 20:

Sequential Result: 2432902008176640000

Parallel Result: 2432902008176640000

Execution Time:

Sequential Time: 0.000000 seconds

Parallel Time: 0.003116 seconds

Speedup: 0.000101

6 . REFERENCES:

• Travis E. Oliphant, *Guide to NumPy*, USA: Trelgol Publishing, 2006.

• John D. Hunter, *Matplotlib: A 2D Graphics Environment*, Computing in Science & Engineering, Vol. 9, No. 3, 2007.

• Michael T. Heath, *Scientific Computing: An Introductory Survey*, 2nd Edition, McGraw-Hill, 2002.

• Rajkumar Buyya, *High Performance Computing: Paradigm and Infrastructure*, Wiley Press.

• Sheldon M. Ross, *Simulation*, 5th Edition, Academic Press, 2012.